

Working with PSC Bridges-2

The federal government has kindly given us some GPU computing credits so that you all can run your code on GPUs. This document describes how to:

1. Make an account.
2. Connect to Bridges-2.
3. Set up your Bridges-2 environment.
4. Work with Bridges-2.

Making an account

Our resource allocation is managed by NSF ACCESS, a program which gives access to several high-performance computing platforms. To get access to Bridges-2, you will first need to make an account with NSF ACCESS. You can do so here:

<https://operations.access-ci.org/identity/new-user>

It is probably easiest to register with your Berkeley login. Click “Register with an existing identity”, then, on the next page, search for and select “University of California, Berkeley” under “Select an Identity Provider”. Click “LOG ON” and follow the steps to make an account.

Once you have received an account, **send your GSI an email with your ACCESS username**. We will add you as a user on our ACCESS allocation. If you are curious, you can look at the allocation by logging in at <https://allocations.access-ci.org/>.

Sometime after we add you to the allocation, probably within 1 business day, the Pittsburgh Supercomputing Center will send you an email with account info. They will tell you how to set your PSC password.

Connecting to Bridges-2

(This information is condensed/adapted from the Bridges-2 User Guide)

Now that you have a PSC account, you can connect to Bridges-2. Bridges-2 contains two broad categories of nodes: compute nodes, which handle the production research computing, and login nodes, which are used for managing files and environments, submitting batch jobs and launching interactive sessions. **Do not do research computing on login nodes.**

One simple way to connect is via ssh. You can ssh to Bridges-2 with `ssh psc_username@bridges2.psc.edu` and put in the password for your PSC account. This will connect you to a “login node”.

Another way to connect is via their online interface, called Open OnDemand. It has a nice file browser and is where you will run jupyter notebooks. Please read the instructions later in this file before running anything :)

Setting up your Bridges-2 environment

Now you will need to set up your Bridges-2 environment for GPU computing. Please follow these steps:

1. SSH into a login node: `ssh psc_username@bridges2.psc.edu`
2. The login environment is pretty bare-bones, and doesn't contain Conda. Load a "module" provided by PSC which contains Conda using `module load anaconda3`.
3. Create your own Conda environment containing PyTorch using the environment specification we provide: `conda env create -f /ocean/projects/mth250011p/shared/215a/lab4/code/environment.yaml`
4. Activate your new Conda environment: `conda activate env_215a`
5. Install an IPython kernel corresponding to your new Conda environment (so that you can use it in Jupyter notebooks): `python -m ipykernel install --user --name env_215a --display-name env_215a`
6. Clean up some conda files to save space: `conda clean --all`, when prompted, just enter y.

Hopefully there were no issues in these steps! If you ever seriously mess up your environment, you can just delete it (`conda env remove -n env_215a`) and re-do these steps.

Testing

Let's test out that our environment works. If any of the following steps fail, check again that you correctly followed all of the instructions! If you did, please contact the GSI.

First, check that the PyTorch installation works:

1. From the login node, create an interactive 10-minute GPU job with `interact -gpu -t 00:10:00`. Wait for your command-line to appear within that job.
2. Run `nvidia-smi`. It should give you some information about the GPU.
3. Run `module load anaconda3` so that you can use Conda again.
4. Activate your 215a environment with `conda activate env_215a`
5. Create a Python shell by running `python`
6. Check that you can import PyTorch: `import torch`
7. Check that PyTorch has some idea of its CUDA version: `print(torch.version.cuda)`. This should print some number like 12.1.
8. Check that PyTorch is able to use CUDA (i.e. able to use a GPU): `torch.cuda.is_available()`. This should return True.

Now, check that the jupyter kernel was properly installed:

1. Log in to the Bridges-2 web interface
2. Click on the app "Jupyter Lab: Bridges2"

3. Enter 1 for the number of hours, 1 for the number of nodes, `mth250011p` for the account, and **GPU-shared for the partition**. You should **never use the GPU partition** unless you have a good reason (it has 8x the number of GPUs but also costs us 8x as much!). Enter `--gpus=1` under Extra Slurm Args. Click “Launch”
4. On the next page, wait for the server to start. It will say “Queued”, then maybe “Starting”, and then will say “Running”. Once the server is running, click the button that says “Connect to Jupyter”.
5. A JupyterLab window will open. Under “Notebook” you should see an option which says “env_215a”. This will create a notebook running the 215a Conda environment. Click on that button.
6. Check that the notebook is using the right Python executable corresponding to your env_215a environment: `import sys` then `print(sys.executable)`. The path should be something like `/jet/home/your_username/.conda/envs/env_215a/bin/python`.
7. Click the blue “+” button, scroll down a bit, and make a Terminal. In the terminal, run `nvidia-smi`. You should see **one** GPU! If you see more or fewer GPUs, you did not put in the right settings in step 3.
8. Go back to the notebook. Check that you can import PyTorch: `import torch`
9. Check that PyTorch has some idea of its CUDA version: `print(torch.version.cuda)`. This should print some number like 12.1.
10. Check that PyTorch is able to use CUDA (i.e. able to use a GPU): `torch.cuda.is_available()`. This should return `True`.
11. Hopefully everything has worked correctly. In the top left, click “File” then “Shut Down”. Back on the web dashboard for interactive sessions, you should see your Jupyter Lab session say “Completed”.

Working with Bridges-2

You will encounter storage and computing limits on Bridges-2 because they have finite resources. Here are some details about storage:

- Your `$HOME` directory has only 25GB of space. You can run the command `my_quotas` on a login node to see your usage.
- We have a shared `$PROJECT` directory (`/ocean/projects/mth250011p`) with a total of 50GB of space. We will put shared datasets, files, etc. under `$PROJECT/./shared`. If you are running out of space in `$HOME`, please don’t put too much stuff in `$PROJECT` without confirming with us.

The most important point is that **our allocation of GPU-hours is finite and shared with the whole class**. You can see how many hours are remaining for the whole class at any moment by running the `projects` command on a login node. Each GPU-hour costs the government roughly \$0.35. Many of the instructions below will be about how to efficiently use these limited resources.

The Bridges-2 GPU nodes have 8 GPUs per node. **For much of this class,**

you will only need one GPU! Bridges-2 has two partitions to which jobs can be submitted: GPU and GPU-shared. GPU will allocate one or more whole nodes with 8 GPUs each, thus using 8 or more GPU-hours per hour of running. GPU-shared allows sharing a node, using only 1 or 2 or 3 (etc.) GPUs and using only 1 or 2 or 3 GPU-hours per hour of running. **Do not run any jobs on GPU unless you really need 8 GPUs.**

Note that **you can only use Bridges-2 for work directly related to this class.** Using our resources for other purposes violates the terms of our NSF allocation.

Running a job on 1 GPU

Most of your computing will probably be done by submitting jobs. Jobs should almost always be submitted to the GPU-shared partition, requesting 1 GPU. Here is an example job to test things out and print some diagnostics. Lines starting with #SBATCH contain options passed to the Slurm job scheduler. Please pay careful attention to them—you will need to set them correctly when you make your own jobs.

```
#!/bin/bash

# our script starts with a bunch of Slurm options

#SBATCH --account=mth250011p      # don't change this
#SBATCH --job-name=my_cool_job
#SBATCH --cpus-per-task=5         # GPU-shared allows max 5 cpus per GPU
# we will only allocate 10 min for this job
#SBATCH --time 00:10:00
#SBATCH -o test.out              # write job console output to file test.out
#SBATCH -e test.err              # write job console errors to file test.err

#SBATCH --partition=GPU-shared    # don't change this unless you need 8 GPUs
#SBATCH --gpus=1                  # don't increase this unless you need more than 1 GPU
# (can instead specify --gpus=v100-16:1 or --gpus=v100-32:1 to specifically
# request a 16GB or 32GB GPU)

# let's now print some diagnostic info

echo "hello world!"
echo "The time is now $(date)"
echo "The ID of this job is $SLURM_JOB_ID"
echo "The job is currently in directory $(pwd)"

# print GPU info
echo "Here is some info about the GPU the job has access to:"
nvidia-smi
```

```

# now let's do the actual computing part.
# in this example we will just test that PyTorch is working correctly.
#####
# first load a module allowing us to use Conda
module load anaconda3

# activate your personal 215a environment
conda activate env_215a

# check which python executable you're running
echo "The python executable in this environment is:"
which python

# run a short script to test pytorch GPU access
echo "Torch's CUDA version and CUDA availability:"
python -c "import torch; print(torch.version.cuda); print(torch.cuda.is_available())"

# note: if we were running an actual Python script we'd just run:
# python my_cool_script.py
#####

echo "The job is done!"
echo "The time is now $(date)"

```

If you have written this job as `test.sh`, you can submit it from a Bridges-2 login node with `sbatch test.sh`. Before you write your own jobs, try running this example job. You should see the output in `test.out`.

At any moment, you can see the status of your running jobs with the command `squeue -u your_username`. The column `ST` indicates the status of your jobs. `PD` means that it is pending (waiting in the queue). `R` means that it is running.

Interactive shell environment with a GPU

Sometimes you won't need a full jupyterlab environment, but you will still need to interactively develop/experiment on a GPU node (e.g. step through the commands in your job script one-by-one to work out where things are going wrong).

You can obtain an interactive session on the GPU-shared partition with 1 GPU using the command `interact -gpu -t 01:00:00`, where we have also specified a 1-hour time limit.

For example, we used such an interactive session when we set up the environment to test that it works correctly.

Efficiently using the Bridges-2 jupyterhub

Most of your computing will probably be done by submitting jobs, and most of your development/experimentation will probably happen locally (without a GPU). However, if you need to interactively experiment with a GPU, the Bridges-2 jupyterhub is a good option.

On the web interface, click “Jupyter Lab: Bridges2”. Here are the parameters to set:

- Number of hours: however long you will actively be using the server. So probably not longer than 6 hours.
- Number of nodes: 1
- Account: `mtb250011p`
- Partition: **GPU-shared** (not GPU!!! GPU will take a whole node and will cost 8 GPU-hours per hour that it runs. GPU-shared lets you get just 1 GPU.)
- Extra Slurm Args: `--gpus=1`.

Click “Launch”. On the next page, wait for the server to start. It will say “Queued”, then maybe “Starting”, and then will say “Running”. Once the server is running, click the button that says “Connect to Jupyter”. A JupyterLab window will open.

Once you are done working, please shut down the jupyterlab server. Two options:

- On the jupyterlab server, click File -> Shut Down.
- In the web interface, click the button to the right of “Interactive Apps”, which should take you here. You will see a running jupyter lab server, and can just click to stop it.

Please don’t leave your server running idle overnight! And for any actual computation (i.e. not just development), like training a NN, please submit jobs instead of using jupyterlab.

Avoiding wasting GPU resources

One of the easiest ways to prevent wasting GPU resources is to follow a better development workflow. Here is our suggested workflow:

1. Develop all code locally (on your own computer). Never edit any code in an editor on Bridges-2.
2. Test code locally. If you don’t have access to GPUs, you probably can’t run your full intended job on a CPU. But you can still test a shorter run, smaller dataset, etc. on a CPU. Make sure that everything loads properly and that your results save in the format you want them.
3. Once you are confident your code will work as you intend, somehow transfer the code to Bridges-2 (my suggestion: use a private or public github repo; push to it from your laptop, and pull from Bridges-2).
4. Submit the job on Bridges-2.

This way, most of your GPU resource usage will be for actual production runs, rather than testing or debugging. The main possible hangup will be how PyTorch transfers information between CPU and GPU, which won't be tested. But such issues should come up right in the beginning of a run, so you won't use much GPU time. Another benefit of this workflow is that you will only ever have one updated version of your code, and you won't have to deal with moving changes which you made on Bridges-2 back to your local machine.

Some more guidelines:

1. Give your jobs a reasonably short time limit.
2. Make sure the jobs actually terminate when you get what you need (e.g. don't run NN training for hours and hours after it converges).
3. Don't leave jupyterlab servers or interactive sessions running idle!